ADVANCED PHASE | SESSION 21 OF 30

Day 21

Data & NumPy

2 hours	3 concept files	28 problems	1 PDF guide

Topics: ndarray vs list • array creation • broadcasting • slicing & indexing • vectorized operations • np.linalg basics

Session Overview

Goal: Introduce NumPy as the foundation of Python's data-science stack. Students learn why ndarray outperforms plain lists, how to create and inspect arrays, how broadcasting eliminates manual loops, how to slice and index arrays (including 2D and boolean indexing), and the basics of linear algebra with np.linalg.

Outcome: By the end of this session, students can confidently create NumPy arrays, perform vectorized calculations on real Indian e-commerce/fintech-style data (Swiggy, Zomato, Flipkart, Paytm, IRCTC, BigBasket), apply broadcasting rules, slice/index multi-dimensional arrays, and solve small systems of linear equations — setting up directly for Day 22's Pandas DataFrames.

Session Timeline

Time	Segment	Deliverable
0:00 - 0:10	Why NumPy? Lists vs Arrays (discussion)	-
0:10 - 0:35	Array creation & attributes	day21_01_numpy_arrays_basics.py
0:35 - 0:55	Vectorized ops & performance demo	day21_01_numpy_arrays_basics.py
0:55 - 1:05	Break	-
1:05 - 1:35	Broadcasting rules + row/column vectors	day21_02_broadcasting_indexing.py
1:35 - 1:55	Slicing, fancy & boolean indexing	day21_02_broadcasting_indexing.py
1:55 - 2:20	np.linalg: dot, matmul, det, inv, solve	day21_03_linalg_basics.py
2:20 - 2:40	Guided examples walkthrough	All 3 concept files
2:40 - 3:00	Practice problems + wrap-up	day21_practise_problems_*.py

TIP:
This is a math-adjacent session. Some students may feel intimidated by broadcasting and linear algebra terms. Anchor every concept in a concrete rupee/order/seat example before showing the abstract rule.

Prerequisites & Preparation

Requirements

Requirement	Details
Python	3.12, with virtual environment active (uv)
Package	numpy - install: pip install numpy --break-system-packages
Prior Days	Day 6 (lists), Day 15 (functional patterns) recommended
Editor	VS Code with Python + Pylance extensions
Files ready	day21_01/02/03 concept files, 3 practice files

Instructor Pre-Session Checklist

[ ] Confirm numpy is installed in every student's virtual environment

[ ] Pre-run all 3 concept files once to catch any environment quirks

[ ] Have the performance-comparison snippet ready to run live (Section 5, file 1)

[ ] Prepare a whiteboard/slide showing the broadcasting rule diagram

[ ] Review the 4 guided examples per file so you can extend them live

Step-by-Step Instruction Guide

STEP 1 - Why NumPy? Lists vs Arrays

"Today we're starting the Data phase of the course. Everything you've learned — loops, functions, OOP — has been about general-purpose programming. NumPy is a library built for ONE thing: fast math on large amounts of numbers. Let's see why that matters."

day21_01_numpy_arrays_basics.py

```
import numpy as np

swiggy_order_prices = [249, 399, 180, 550, 275]
swiggy_prices_arr = np.array(swiggy_order_prices)
print(swiggy_prices_arr)
[249 399 180 550 275]
```

Common Errors

Error	Cause	Fix
ModuleNotFoundError: numpy	Package not installed in venv	pip install numpy --break-system-packages
TypeError: unsupported operand	Mixing list and array in arithmetic	Convert list to array first with np.array()

DONE WHEN:
Students can run `type(swiggy_prices_arr)` and see `<class 'numpy.ndarray'>` printed, confirming they've created a real NumPy array, not a list.

STEP 2 - Array Creation & Attributes

"NumPy gives us several shortcuts for creating arrays without typing out every value: zeros, ones, arange, and linspace. Let's try each."

day21_01_numpy_arrays_basics.py

```
delivery_day_numbers = np.arange(1, 11)
discount_levels = np.linspace(0, 50, 6)
print(delivery_day_numbers)
[ 1  2  3  4  5  6  7  8  9 10]
print(discount_levels)
[ 0. 10. 20. 30. 40. 50.]
```

Point out the four key attributes every array carries: **shape** (dimensions), **ndim** (number of axes), **size** (total elements), and **dtype** (the single data type shared by all elements).

WATCH OUT:
np.arange(start, stop, step) EXCLUDES the stop value, just like range(). np.linspace(start, stop, num) INCLUDES both endpoints. This is the single most common student mix-up in this section.

STEP 3 - Vectorized Operations & Performance

"This is the payoff. Watch what happens when we apply GST to an entire array of Flipkart prices in ONE line, no loop required."

day21_01_numpy_arrays_basics.py

```
flipkart_prices = np.array([1200, 3400, 899, 15999, 599])
prices_with_gst = flipkart_prices * 1.18
print(prices_with_gst)
[ 1416.    4012.    1060.82 18878.82   706.82]

above_1000 = flipkart_prices > 1000
print(flipkart_prices[above_1000])
[ 1200  3400 15999]
```

Live-run the loop-vs-vectorized performance comparison (Section 5 of the file) so students see real timing numbers on their own machines. Emphasize this is why NumPy underlies Pandas, scikit-learn, and virtually every data-science tool in Python.

DONE WHEN:
Students should be able to explain, in their own words, why flipkart_prices[above_1000] returns only the elements where the mask is True. This 'boolean masking' pattern reappears constantly in Pandas.

STEP 4 - Broadcasting Rules

"Broadcasting is how NumPy applies an operation between arrays of DIFFERENT shapes. The rule: compare shapes from the RIGHT — dimensions must match, or one of them must be 1."

day21_02_broadcasting_indexing.py

```
zomato_weekly_orders = np.array([[120,135,110,140],
                                 [80, 95, 70, 100],
                                 [200,210,190,220]])
weekly_platform_fee = np.array([5, 5, 8, 8])
print(zomato_weekly_orders - weekly_platform_fee)
[[115 130 102 132]
 [ 75  90  62  92]
 [195 205 182 212]]
```

Draw the shapes on the whiteboard: (3,4) and (4,) — the trailing dimension 4 matches, so the 1D array is treated as if repeated across all 3 rows. Then show the column-vector case using `.reshape(-1, 1)` for per-row adjustments.

Error	Cause	Fix
ValueError: operands could not be broadcast	Shapes incompatible (mismatched trailing dims)	Check shapes with .shape; reshape one array
Wrong axis affected	Forgot .reshape(-1, 1) for column vector	Reshape 1D array to (n, 1) for row-wise broadcast

STEP 5 - Slicing, Fancy & Boolean Indexing

"Indexing in NumPy extends everything you know about list slicing to multiple dimensions, plus two new tricks: fancy indexing and boolean indexing."

day21_02_broadcasting_indexing.py

```
irctc_fare_matrix[0, 2]        # single element
irctc_fare_matrix[1, :]        # entire row
irctc_fare_matrix[:, 0]        # entire column
irctc_fare_matrix[[0, 2]]      # fancy: rows 0 and 2
irctc_fare_matrix[irctc_fare_matrix > 2000]  # boolean
```

WATCH OUT:
A basic slice (e.g. arr[:3]) returns a VIEW — modifying it modifies the original array! Fancy indexing (arr[[0,2]]) always returns a COPY. This distinction trips up even experienced Python developers.

STEP 6 - np.linalg Basics

"Last piece for today: linear algebra. You won't need heavy math theory — just enough to compute weighted totals (dot products), transform matrices, and solve simple equations."

day21_03_linalg_basics.py

```
coefficients = np.array([[2, 1], [1, 3]])
totals = np.array([8, 9])
combo_counts = np.linalg.solve(coefficients, totals)
print(combo_counts)
[3. 2.]
```

Walk through the ration-shop / combo-pack word problems from the file step by step: translate the word problem into a coefficient matrix and a totals vector, then call np.linalg.solve(). Avoid deriving the algebra formula by hand — the goal is API fluency, not manual algebra.

DONE WHEN:
Students should be able to compute a dot product with np.dot() or @, multiply two matrices with @, and call np.linalg.solve() correctly on a 2x2 system without instructor help.

Session Wrap-Up

Student Checklist

[ ] I can explain why ndarray is faster than a Python list for numeric work

[ ] I can create arrays with np.array, np.zeros, np.ones, np.arange, np.linspace

[ ] I can read shape, ndim, size, and dtype from any array

[ ] I can apply broadcasting rules to add a row vector or column vector

[ ] I can slice, fancy-index, and boolean-index both 1D and 2D arrays

[ ] I can compute a dot product, matrix product, determinant, and solve a linear system

Key Concepts Recap

Concept	Key API
Array creation	np.array(), np.zeros(), np.ones(), np.arange(), np.linspace()
Attributes	.shape, .ndim, .size, .dtype
Vectorized ops	arr * scalar, arr1 + arr2, arr.sum()/.mean()/.max()/.min()/.std()
Broadcasting	Row vector (n,), column vector via .reshape(-1, 1)
Indexing	arr[:3], arr[1, :], arr[[0,2]], arr[arr > x]
Linear algebra	np.dot(), @, .T, np.linalg.det(), .inv(), .solve()

Next Day Preview

Day 22 - Data & Pandas: DataFrame and Series objects, read_csv()/to_csv(), groupby() aggregations, merge()/join(), and basic plotting with df.plot(). Pandas is built directly on NumPy, so every concept from today (vectorization, broadcasting, boolean masking) carries forward immediately.

> **TIP:**
> Troubleshooting note: if a student's array prints with unexpected extra decimals or dtype changes after an operation, remind them that NumPy upcasts int arrays to float automatically when division or GST-style multiplication is involved. This is expected behavior, not a bug.

Quick Reference Card

Daily Workflow Commands

```
source .venv/bin/activate      # activate virtual env (mac/linux)
.venv\Scripts\activate         # activate virtual env (windows)
pip install numpy --break-system-packages
python day21_01_numpy_arrays_basics.py
python day21_02_broadcasting_indexing.py
python day21_03_linalg_basics.py
```

VS Code Shortcuts

Shortcut	Action
Ctrl/Cmd + `	Toggle integrated terminal
F5	Run/debug current file
Shift + Alt + F	Format document
Ctrl/Cmd + /	Toggle line comment

Tool Summary

Tool	Purpose
numpy	N-dimensional arrays, vectorized math, linear algebra
np.arange / linspace	Generate sequences of numbers
np.linalg	Dot products, matrix ops, determinants, solving systems
Boolean masking	Filter arrays by condition, e.g. arr[arr > x]